

Master d'informatique - 1
Architecture des Systèmes Intégrés et Optimisations
Décembre 2010 – Documents autorisés – Durée 2h

Utiliser impérativement les feuilles quadrillées fournies pour les schémas

Les réponses non justifiées seront considérées comme fausses

L'objectif de cet exercice est de comparer la performance d'exécution d'un algorithme de traitement d'images sur deux réalisations différentes du Mips. La première est la réalisation classique sur un pipeline de 5 étages présenté en cours. Cette réalisation est appelée **Mips**. La seconde, appelée **SS2**, est une réalisation superscalaire à deux pipelines. Il s'agit de la réalisation à 5 étages présentée en cours. **SS2** à la même fréquence d'horloge que la réalisation **Mips**.

Une image est composée d'un ensemble de pixels organisés sous forme d'un tableau. Dans une image en couleur, chaque pixel est une combinaison de 3 nuances de couleur: rouge, verte et bleue. Un pixel en couleur est donc codé sur un entier (4 octets). L'octet de poids fort n'est pas utilisé. Chacun des 3 autres octets représentent une composante de couleur du pixel.

Une fonction de traitement d'images consiste à appliquer un certain algorithme à un tableau de pixels. Dans cet exercice, on étudie une fonction qui permet de séparer les composantes d'une image en couleur pour obtenir 3 images : une image pour la composante rouge, une pour la verte et une pour la bleue. Les images obtenues par cette décomposition sont donc des images monochromes où chaque pixel est codé sur 1 octet.

```
void DecomposeImage (int  *img      ,
                     char *red_img,
                     char *grn_img,
                     char *blu_img,
                     int  size      )
{
    unsigned int;

    for (i=0 ; i!=size ; i++)
    {
        red_img [i] = img [i] >> 16;
        grn_img [i] = img [i] >>  8;
        blu_img [i] = img [i]      ;
    }
}
```

Le code de la boucle principale de cette fonction en assembleur Mips pipeline est le suivant.

```
Loop :      Lw      r10, 0 (r4 )
            Srl      r11, r10, 8
            Srl      r12, r10, 16

            Sb      r12, 0 (r5 )
            Sb      r11, 0 (r6 )
            Sb      r10, 0 (r7 )

            Addiu   r5 , r5 , 1
            Addiu   r6 , r6 , 1
            Addiu   r7 , r7 , 1
            Addiu   r4 , r4 , 4
            Bne     r4 , r8 , Loop
            Nop
```

A l'entrée de la boucle, le registre R4 contient l'adresse du premier pixel de l'image en couleur **img**, R5 l'adresse de l'image rouge, R6 l'adresse de l'image verte et R7 l'adresse de l'image bleue. R8 contient l'adresse de fin de l'image en couleur.

- 1- Analyser l'exécution de la boucle sur la réalisation **Mips**. Il n'est pas demandé de faire un schéma mais, d'indiquer simplement les dépendances de données et les situations qui provoquent les cycles de gel. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 2- Analyser l'exécution de la boucle à l'aide d'un schéma simplifié sur la réalisation **SS2**. On suppose que l'étiquette **Loop** se trouve sur une adresse **alignée** et que le buffer d'instructions est vide au début de l'exécution de la boucle. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 3- Pour la réalisation **Mips**, optimiser le code de la boucle à l'aide de la technique "pipeline logiciel" en indiquant clairement le nombre d'étages et quelles instructions font partie de quel étage. Puis, réordonner afin d'éviter au maximum les cycles perdus. Calculer le nombre moyen de cycles par itération en indiquant simplement les situations qui provoquent les cycles de gel. Calculer le CPI et le CPI-utile.
- 4- Pour la réalisation **SS2**, optimiser le code de la boucle à l'aide de la technique "pipeline logiciel" en indiquant clairement le nombre d'étages et quelles instructions font partie de quel étage. Puis réordonner afin d'éviter au maximum les cycles perdus. Montrer à l'aide d'un schéma simplifié l'exécution de la boucle. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.

Dans la suite, on s'intéresse à la réalisation **Mips**.

Le processeur est relié à la mémoire à travers un cache de données et un cache d'instructions. On suppose que le cache d'instructions est parfait (taux de Miss = 0). On dispose de plusieurs types de cache de données. On souhaite étudier l'influence du cache de données sur la performance d'exécution.

On suppose qu'au début de la boucle le cache de données est vide. L'image couleur contient 1 Kpixels et son adresse est alignée (multiple de 4096). Les images issues de la décomposition sont placées à des adresses consécutives de la mémoire. L'adresse de l'image rouge, **red_img**, est multiple de 4096. Elle contient 1 Koctet. L'image verte se trouve à la suite de l'image rouge et l'image bleue à la suite de l'image verte. Autrement dit :

```
grn_img = red_img + 1024
blu_img = grn_img + 1024
```

1^{er} cas:

Le cache de données est un cache *Write Through* à correspondance directe. Il a une capacité de 4 Koctets. Un bloc du cache contient 16 octets. En cas de *Hit*, il faut 1 cycle pour accéder au cache. En cas de *Miss*, il faut en moyenne 11 cycles pour recevoir le bloc manquant et l'enregistrer dans les mémoires du cache. Puis, il faut 1 cycle supplémentaire pour répondre au processeur

Dans un premier temps, on ne s'intéresse qu'aux lectures en ignorant l'effet des écritures.

- 5- Combien de *Miss* se produisent-ils lors de l'exécution des itérations du code original.
- 6- Calculer le nombre moyen de cycles par itération. Calculer le CPI-utile de la boucle.

On s'intéresse maintenant à l'effet des écritures.

Le cache dispose d'un buffer d'écritures postées d'une seule place. Pour une écriture, lorsque le buffer est libre, il faut 1 cycle pour écrire dans le buffer. Alors le processeur peut continuer à exécuter des instructions et le cache prend en charge l'écriture effective en mémoire. Il faut en moyenne 6 cycles pour effectuer l'écriture et vider le buffer. Lorsque le bloc à modifier n'est pas présent dans le cache, il n'est pas chargé dans le cache (pas de Miss sur écriture).

- 7- Calculer le nombre moyen de cycles par itération en prenant en compte l'effet des lectures et des écritures. Calculer le CPI-utile de la boucle.
- 8- Pour améliorer les performances d'exécution, on décide de dérouler 1 fois la boucle (traitement de 2 pixels à chaque itération). Optimiser le code issu du déroulement en tenant compte du cache de données.
- 9- Combien de *Miss* se produisent-ils lors de l'exécution des itérations du code déroulé et optimisé.
- 10- Calculer le nombre moyen de cycles par itération du code déroulé en prenant en compte l'effet des lectures et des écritures. Calculer le CPI-utile de la boucle.
- 11- Le taux de Miss serait-il différent si, à la place d'un cache à correspondance direct, on utilise un cache associatif par ensemble de 2 blocs.

2^{ème} cas:

Le cache de données est un cache *Write Back* à correspondance directe. Il a une capacité de 4 Koctets. Un bloc du cache contient 16 octets. Sur une écriture, lorsque le bloc n'est pas présent dans la cache, il est chargé dans le cache depuis la mémoire puis, modifié. En cas de *Hit*, il faut 1 cycle pour accéder au cache. En cas de *Miss*, si le bloc à remplacer est *Clean* il faut en moyenne 11 cycles pour recevoir le bloc manquant et l'enregistrer dans les mémoires du cache. Si le bloc à remplacer est *Dirty* il faut en moyenne 16 cycles pour recevoir le bloc manquant et l'enregistrer dans les mémoires du cache. Puis, il faut 1 cycle supplémentaire pour répondre au processeur.

- 12- Combien de *Miss* (distinguer 2 types de Miss) se produisent-ils lors de l'exécution des itérations du code original.
- 13- Calculer le nombre moyen de cycles par itération. Calculer le CPI-utile de la boucle.